

AI Models in Agentic AI: How They Learn, Fail, and Connect to Research Workflows

Jie Hou

NRT Hot Topic: Agentic AI · Lecture 2 · Sep 25, 2026



Materials, demos
& guides

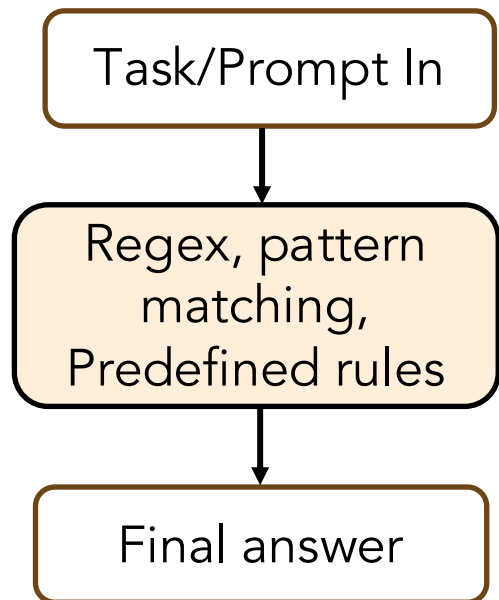
[jiehoulab.github.io/
nrt_agent_week2](https://jiehoulab.github.io/nrt_agent_week2)

From Week 1: AI Agent Workflow

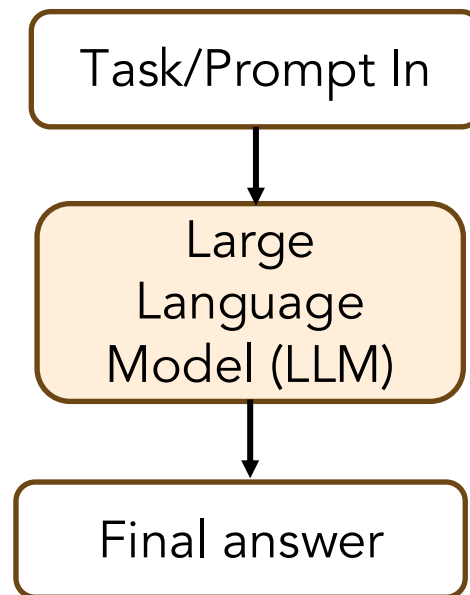
❑ **Definition:** An agent maps what it has seen (h) to what it does (a)

$$\pi : \mathcal{H} \rightarrow \mathcal{A}, \quad a_t = \pi(h_t)$$

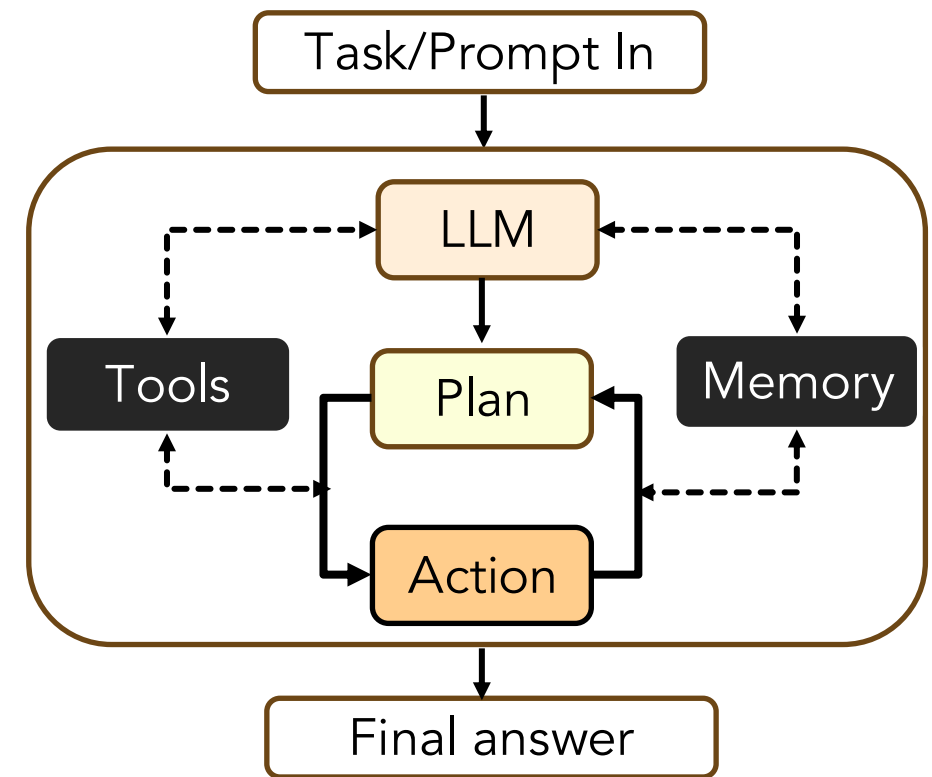
- Rule-based agent (i.e., Symbolic/expert)



- Conversational LLM (e.g., GPT-style chat model)



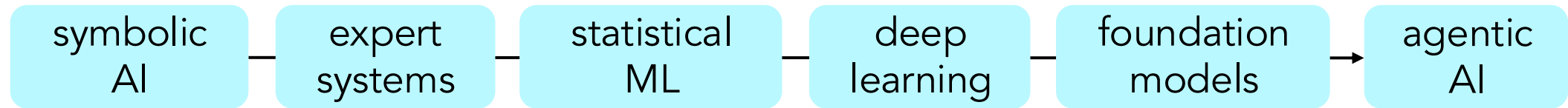
- Tool-augmented LLM Agent (i.e., ChatGPT, Claude)



Today's focus: what happens inside the LLM box, and how we connect it to tools reliably.

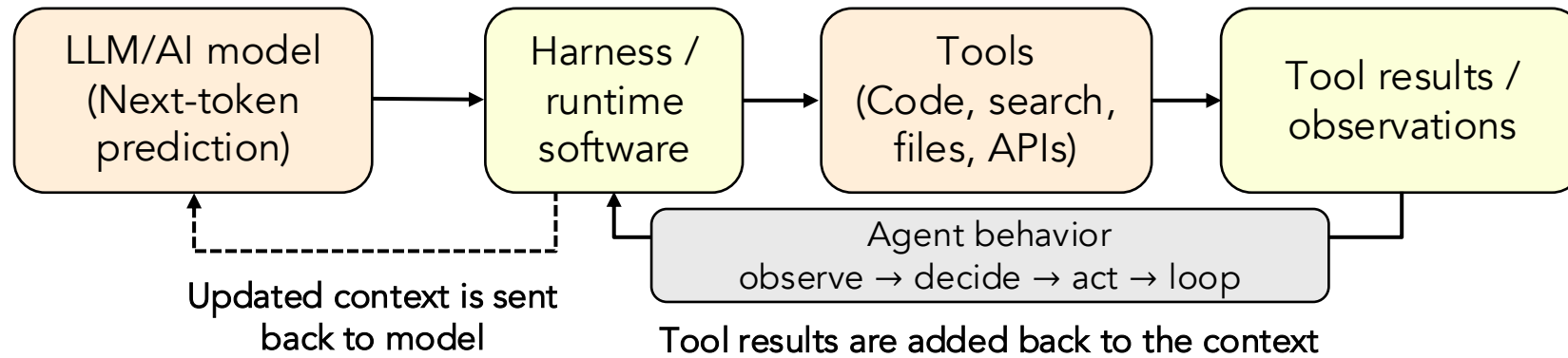
Key Takeaways from Week 1: What is an agent

1. AI has evolved through several major paradigms.



2. An AI model is not the same as an AI agent.

- The AI model generates predictions or outputs that inform decisions.
- An agent adds surrounding software (a harness/runtime) that lets the model use tools, observe results, and continue acting.



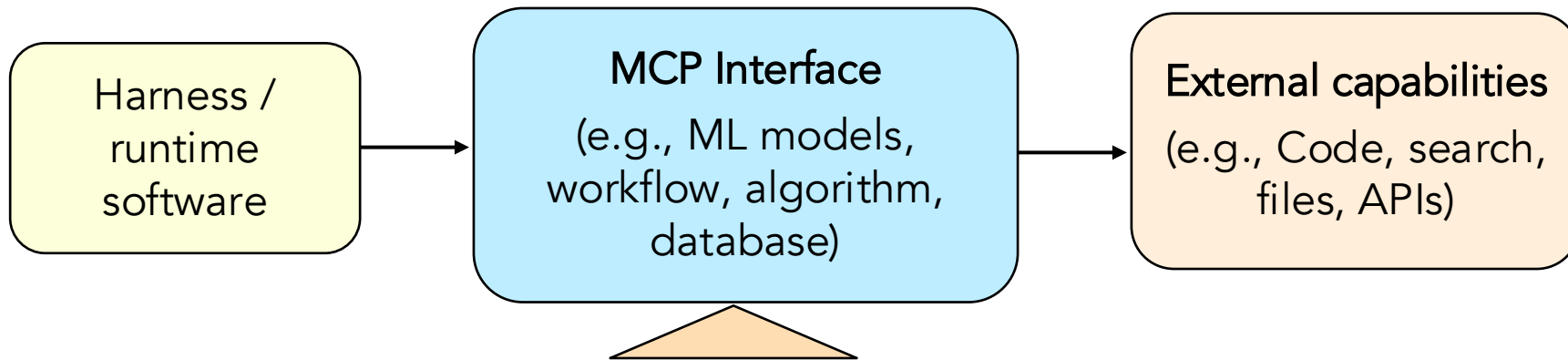
3. Agents operate through an iterative loop. A typical pattern is:

reason/decide → call a tool → observe the result → update context → continue until done.

From Week 1: How Should an Agent Connect to External Capabilities?

❑ **Question:** If the runtime needs to use external tools, how does it discover and call them reliably?

4. Model Context Protocol (MCP) is a common protocol that lets an agent runtime discover and call external tools, resources, and services.



Our focus today: making tool use explicit, validated, and controllable.

Central Message for this lecture

- ✓ **Using an agent means delegating work to an AI system.**

i.e., "Here is my project. Please figure out how it works and help analyze my data."

- ✓ **Designing an agent means deciding what the AI should and should not have to figure out.**

i.e., "Instead of asking the AI to rediscover the workflow from source code, we can define the capability explicitly: inputs, allowed parameters, outputs, validation rules, and permissions."

Our goal today: understand how AI models generate outputs, where they can fail, and how to connect them to research workflows more reliably.



What Do We Mean by "AI Model"?

- ❑ In a supervised-learning setting, an AI model is a learned function that maps inputs to outputs.



- Input: x — features, observations, sequences, images, etc.
- Output: y — labels, scores, predictions, actions, etc.
- Model f_{θ} : a learned mapping from input to output.
- Training data: examples $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_n)$ used to learn parameters θ

$$\hat{f} = \operatorname{argmin}_{f \in \mathcal{H}} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f(x_i), y_i)$$

Training:

Learns the model parameters θ from data by minimizing an objective.

Inference:

Applies the learned model to a new input. This is usually what happens when an agent calls a model.

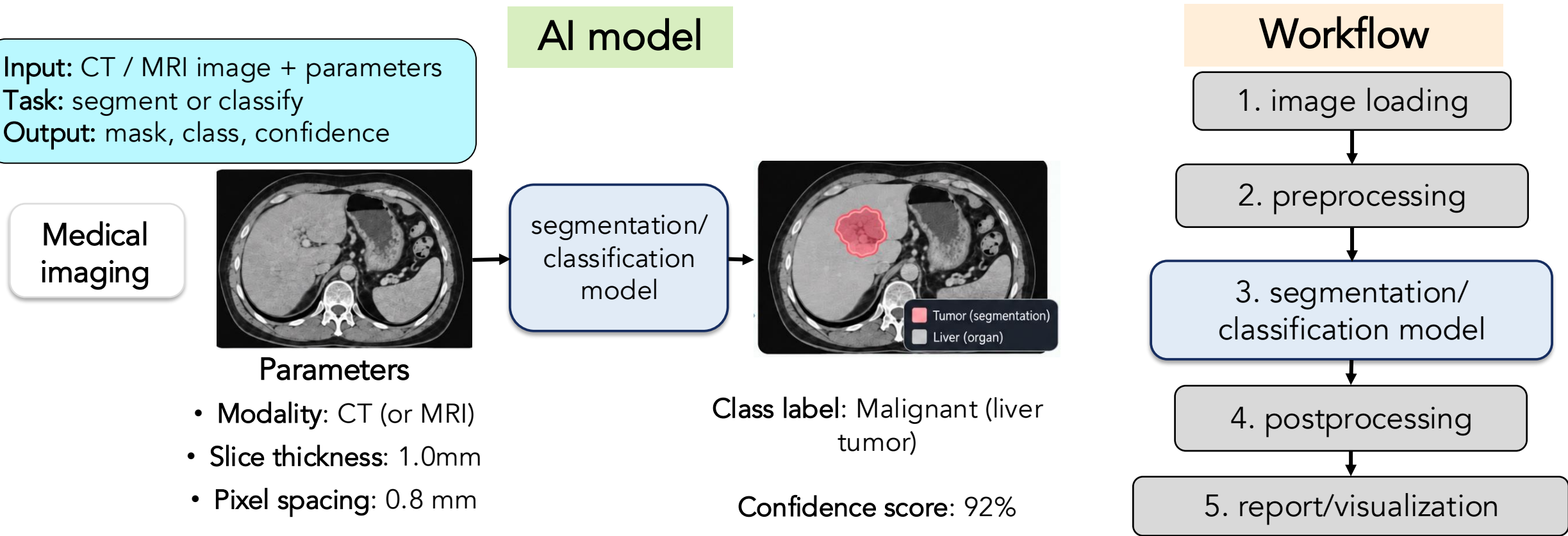
Important distinction

AI model $\neq$ workflow $\neq$ tool

A tool may wrap a model, and a workflow may combine several models and tools.

AI Model vs. Research Workflow: A Medical Imaging Example

- The AI model is the learned component.
- The workflow is the larger process that prepares inputs, calls the model, and interprets the outputs.



Later, we will see how an agent can call either the model or the larger workflow as a capability.

AI Model vs. Research Workflow: A Molecular Structure Example

AI model

Input: protein sequence + optional MSA + parameters
Task: generate or score structure
Output: 3D structure, score, confidence

Molecular
Structure
prediction

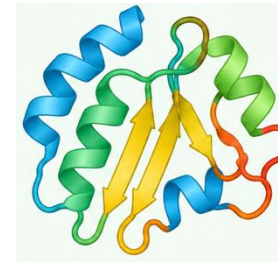
Amino acid sequence

MKTAYIAKQRQISFVKSHFSR
QDILDLIKELGNVTHKGEIPD
.....

Parameters

- num_recycles: 3
- random seed: 42
- use_msa: True

Structure
& Score
predictors



Confidence: 0.92
Global score: 0.88

Workflow

1. Protein sequence

2. MSA/template search

3. Feature generation

4. Structure & Score
predictors

5. Postprocessing

6. Report/visualization

Later, we will see how an agent can call either the predictor itself or the full structure-prediction workflow as a capability.

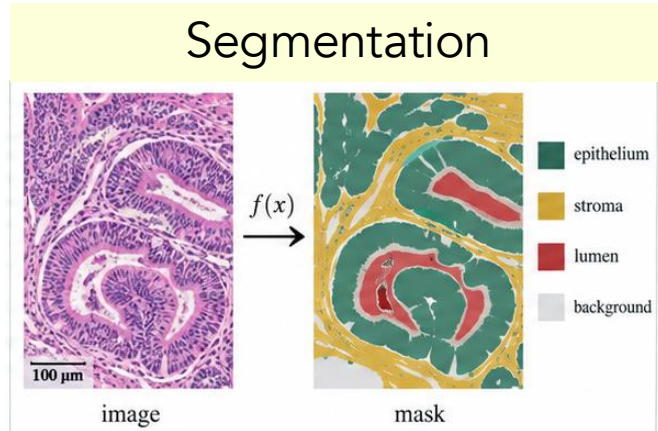
Major Ways AI Models Learn

□ AI models differ in what supervision they receive, what objective they optimize, and what they learn.

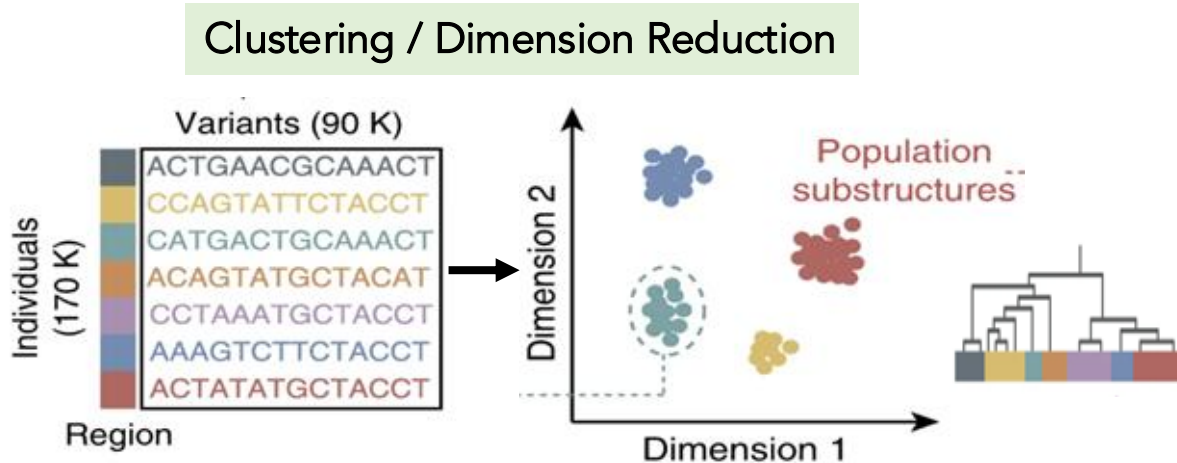
Setup	Observed data	Representative objective	Learned object	Applications
Supervised	Inputs x and labels y : (x_i, y_i)	$\min \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f(x_i), y_i)$	$f_\theta: x \rightarrow y$ or $P_\theta(y x)$	classification, regression, segmentation
Unsupervised	Inputs x_i only	Discover structure without external labels	representations, clusters, or density	clustering, dimensionality reduction, representation learning
Self-supervised	Inputs only; targets are derived from the data itself	Predict a hidden, transformed, or next part of the input	representation or conditional model	masked prediction, next-token prediction, foundation models
Reinforcement	interaction trajectories, actions, rewards	maximize expected cumulative reward	Policies or behavior	decision making, control, game agents

Examples of the Major Learning Paradigms

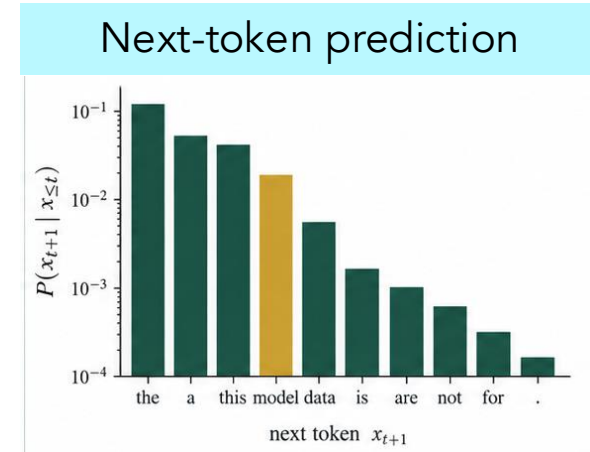
❖ **Supervised:** learn from labeled examples



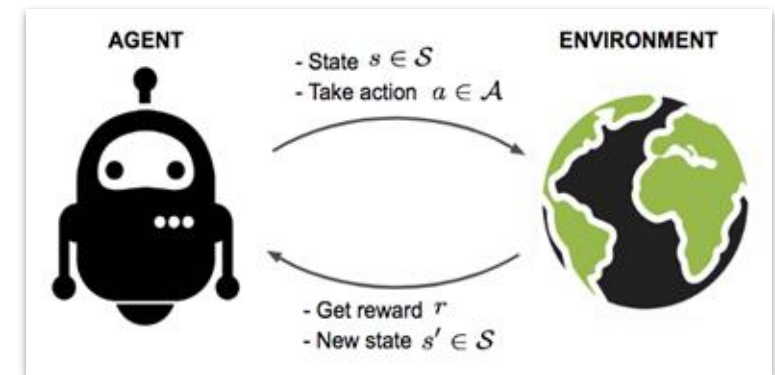
❖ **Unsupervised:** discover structure without labels



❖ **Self-supervised:** create prediction targets from the data itself

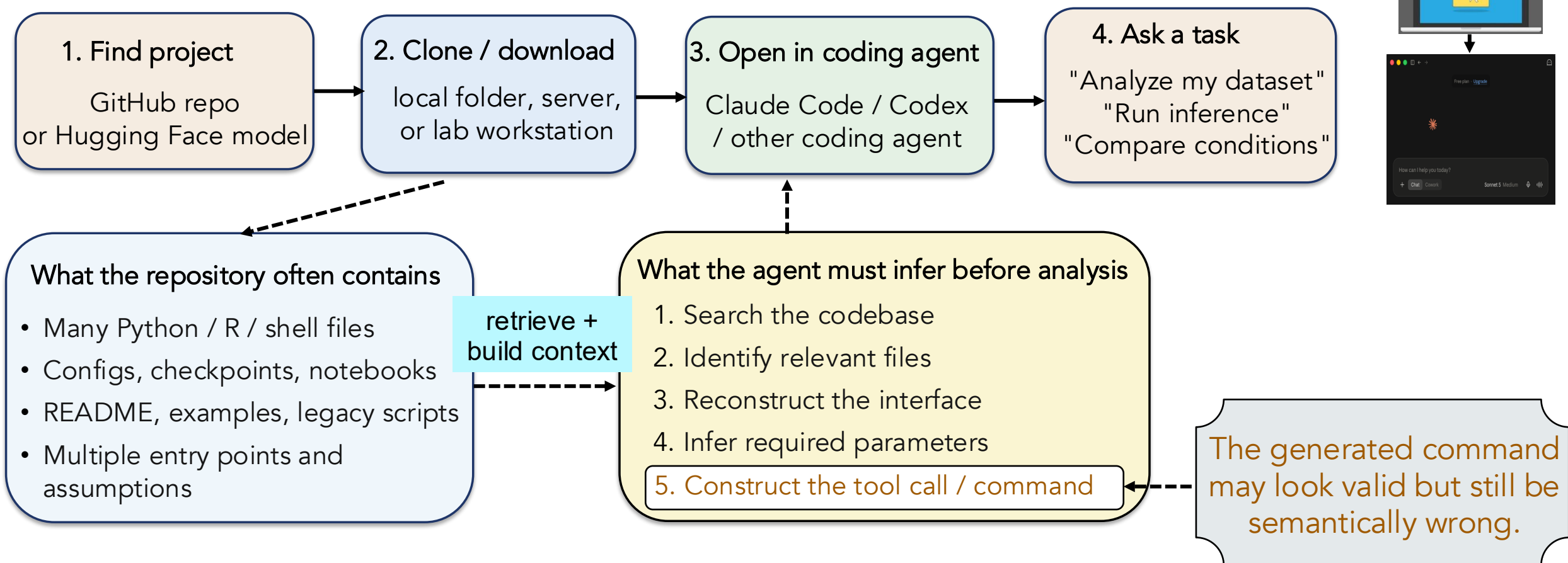


❖ **Reinforcement:** learn from interaction and reward



How Researchers Commonly Use Published AI Projects

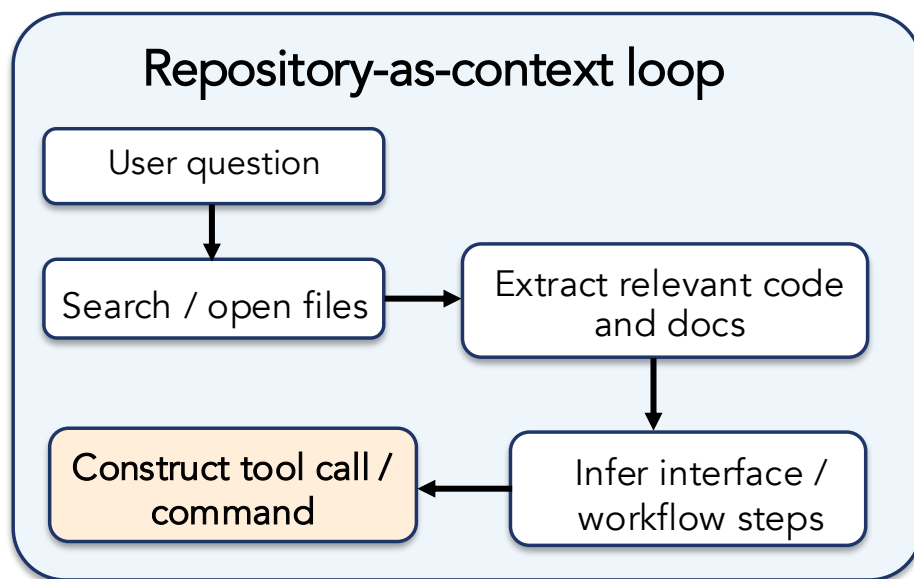
- ❑ **A realistic starting point for researchers:** download an existing GitHub or Hugging Face project and let a coding agent figure it out.



- ❑ **Key point:** the agent is not just running the model — it first has to rediscover how the project is organized and how its interface should be used

Repository-as-Context for a Coding Agent

- ❑ For a new task, coding agents may need to **retrieve** and **reason** over relevant pieces of a large codebase.



```
tissue_analyzer
├── config
│   └── analysis.yaml
├── data
│   ├── case_001.pgm
│   ├── case_002.pgm
│   ├── case_003.pgm
│   └── case_004.pgm
├── models
│   ├── model_card.md
│   └── tissue_rf.json
├── README.md
├── requirements.txt
├── scripts
│   ├── analyze_tissue.py
│   └── classify_tissue.py
```

```
"""
analyze_tissue.py - quantify tissue damage

Parameters may be given on the command
both: command-line values override the
built-in defaults.

Examples:

# everything explicit
python scripts/analyze_tissue.py \
    --slide data/case_003.pgm \
    --magnification 20 \
    --threshold 0.45 \
    --out results/case_003.json
```

Infer how to run
the workflow

- ❑ The agent may need to **spend substantial context and reasoning** on understanding the implementation before performing the analysis.
- ❑ **Reliability** depends on whether the correct files, instructions, and parameters are retrieved and interpreted.

Repository-as-Context: Where Fragility Comes From

❑ Typical failure points for Research tools

- **Context dilution:** Large repositories may contain more relevant information than the model can reliably keep active at once.
- **Competing examples:** Old scripts, aliases, or inconsistent docs can suggest multiple plausible commands.
- **Training prior over exact tool specification:** The model may prefer a common pattern from training, such as **20x** or **--mag** over the exact interface described by your tool.
- **Silent semantic errors:** Some bad commands crash and can be fixed; others run successfully but mean the wrong thing.



1. Type / format mismatch

```
scripts/analyze_tissue.py \  
--slide s1.svs \  
--magnification 20x \  
--out r.json
```

2. Parameter name recall error

```
scripts/analyze_tissue.py \  
--slide s1.svs \  
--mag 20 \  
--out r.json
```

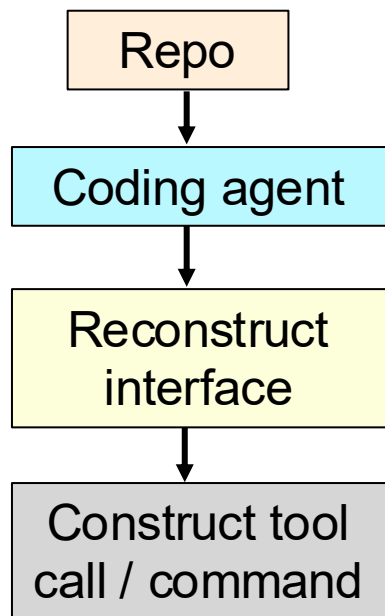
3. Silent scale / range error

```
scripts/analyze_tissue.py \  
--slide s1.svs \  
--magnification 20 \  
--threshold 50 \  
--out r.json
```

Why do these errors happen, and how can we reduce them?

Hands-on Demo 1: Repository as Context

- ❑ **Goal:** watch what the coding agent must infer before it can run the analysis.



Scan for Demo
1 + guide

Agentic AI - Week 2
Michigan State University

DEMO 1 - LEVEL 1 - REPOSITORY AS CONTEXT

Tissue Analyzer + synthetic images

Run the repository directly, then hand the same folder to Claude Code and watch it reconstruct the tool interface from documentation and source alone.

[Download Demo 1](#) [All four demos](#) [Demo 2](#)

Demo1_tissue_analyzer

- config
 - analysis.yaml
- data
 - case_001.pgm
 - case_002.pgm
 - case_003.pgm
 - case_004.pgm
- docs
 - SYNTHETIC_HISTOLOGY_DEMO.md
- models
 - model_card.md
 - tissue_rf_v2.json
 - tissue_rf.json
- previews
 - case_fold_artifact_histology.png
- README.md
- requirements.txt
- results
- scripts
 - analyze_tissue.py
 - classify_tissue.py

Claude Code v2.1.282
Opus 5.5 (1M context) · Claude Team
~/Documents/msu/courses/AI_agent_course/AgenticAI_Course/week2

Updated to latest. Got 13 features, 111 bugfixes, and 52 other changes.
code.claude.com/docs/en/changelog for details

Get to finished work sooner with Opus 5.5. Switch anytime with `/model`.

`> try "refactor <filepath>"`

manual mode on · ? for shortcuts · ← for agents

Problems Output Debug Console **Terminal** Ports

* Baked for 22s · done 4:21 PM

```
> Analyze data/case_mild_injury_histology.pgm
at 20x magnification with threshold 0.45.
```

- Bash(`python3 ../../tools/tissue_analyze.py --slide data/case_mild_injury_histology.pgm --magnification results/case_mild_injury_histology_analyze.json; echo "exit=$?"`)
 - "tool": "tissue_analyze",at 20x magnification with threshold 0.45.
- Bash(`python3 ../../tools/tissue_analyze.py --slide data/case_mild_injury_histology.pgm --magnification`)
 - "tool": "tissue_analyze",
 - "slide": "data/case_mild_injury_histology.pgm",
 - ... +10 lines (ctrl+o to expand)

• I re-ran the analysis and got the same result as last time. About 22% of the tissue is damaged, and t

field	value
Tissue tiles	576
Damage fraction	0.22
Hard edge rate	0.042
Confidence	0.95 (low_confidence: false)
Notes	ok

- **Magnification:** I passed "20x" to the tool as 20, because it rejects the x.
- **Output:** results/case_mild_injury_histology_analyze.json was overwritten with identical contents.
- **Log:** one more line went into runs/unknown.log.

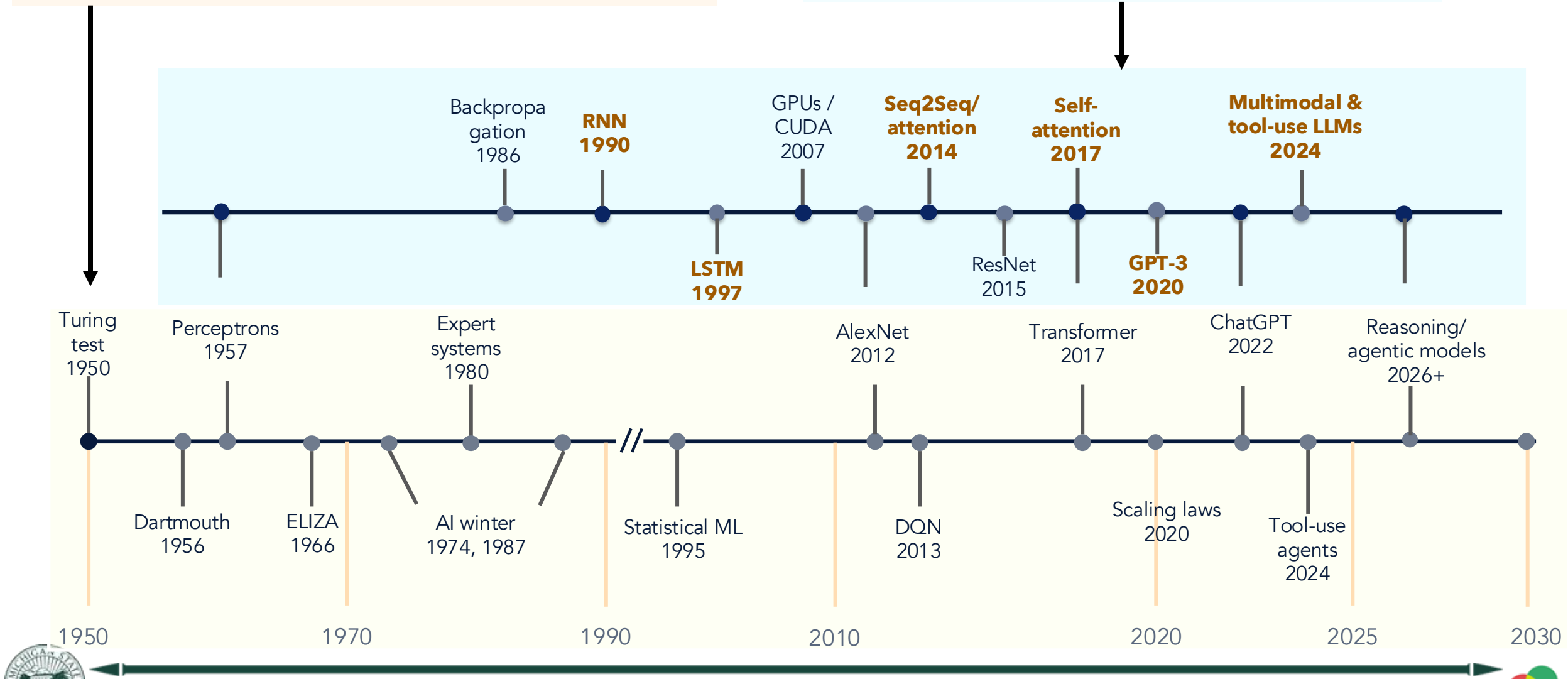
* Worked for 9s · done 4:21 PM



From AI Models to LLM Behavior: Why These Errors Happen

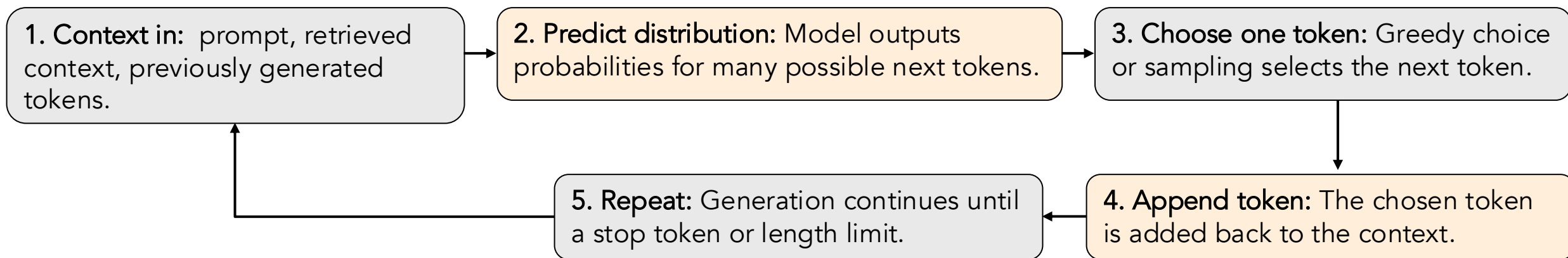
I. Broad evolution of AI systems

II. Key milestones leading to modern LLMs



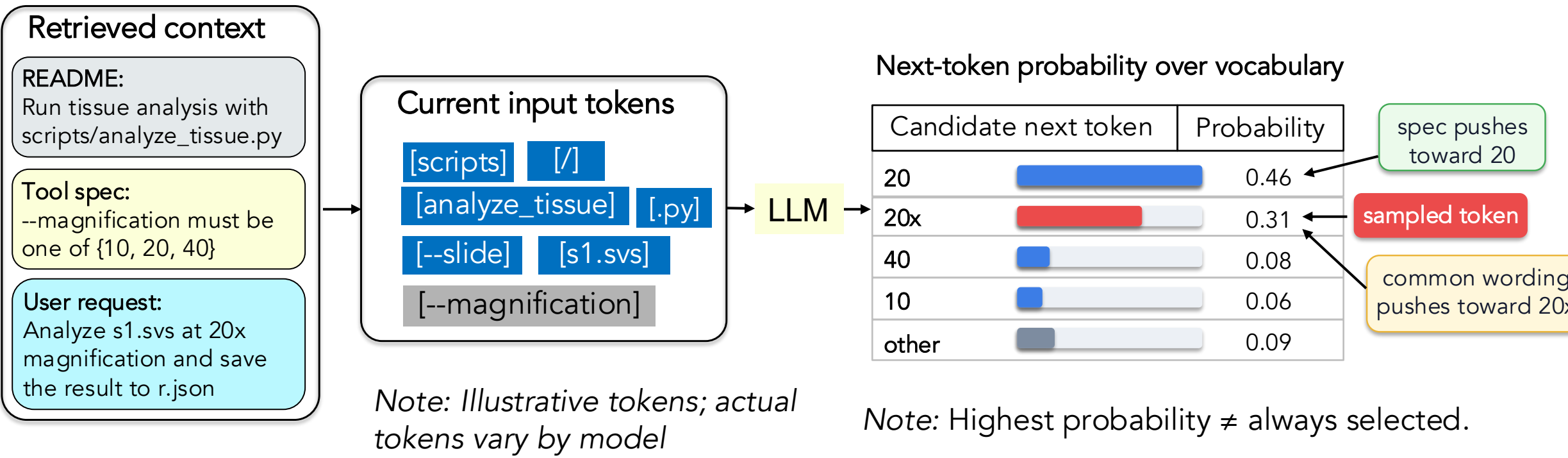
Lesson 1: How Next-Token Prediction Works

- ❑ An LLM repeatedly predicts **a probability distribution over the next token** given its current context and learned training priors, **selects the next token**, appends it to the context, and repeats.



Lesson 1: Why a Wrong Token Can Still Look Plausible

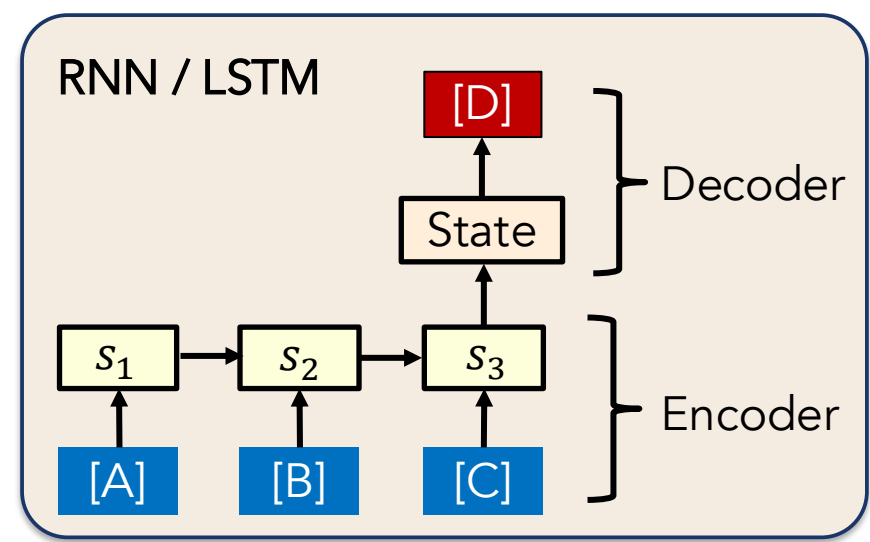
- ❑ An LLM repeatedly predicts a **probability distribution over the next token** given its current context and learned training priors, **select next token**, appends it to the context, and repeats.



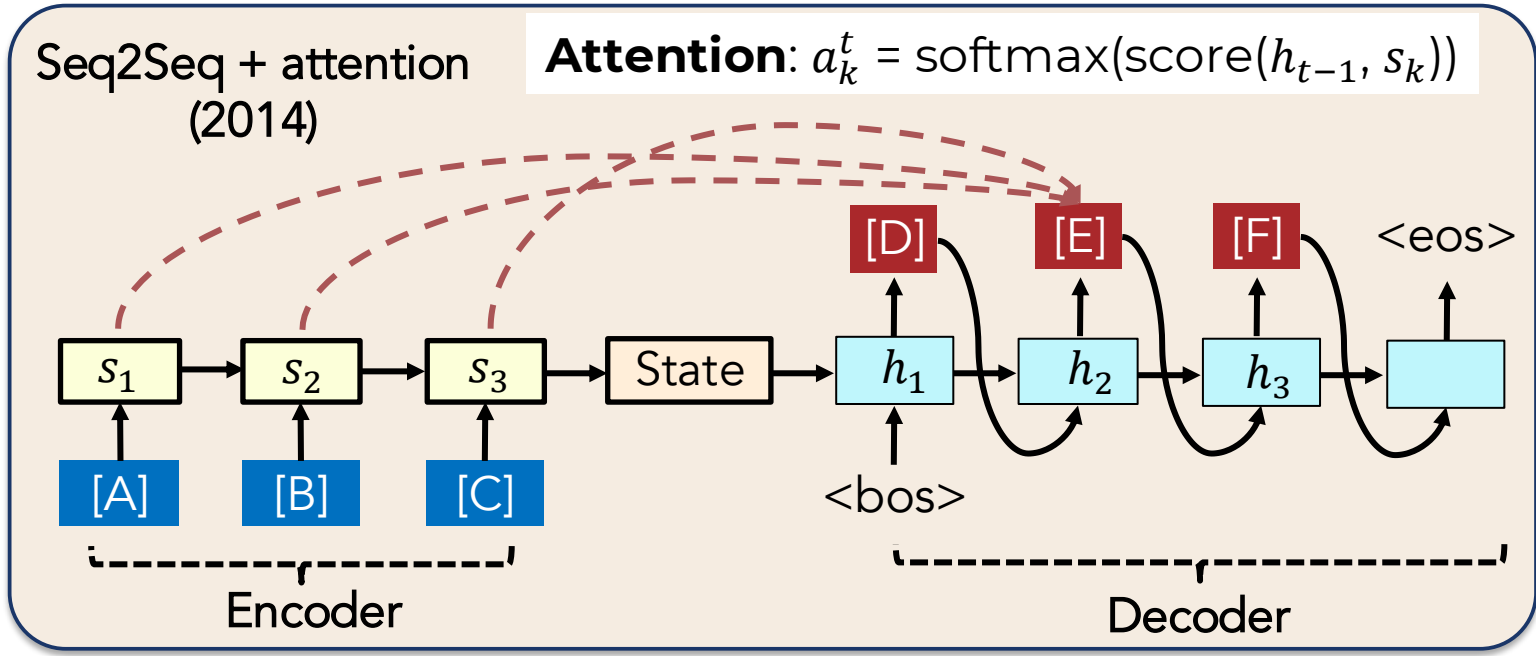
These probabilities are illustrative. If the decoder is sampling rather than choosing greedily, a lower-probability token such as 20x can still be selected.

Lesson 2: From RNN to Attention

- **Encoder**: represents the input sequence as hidden states
- **Decoder**: generates the output sequence from those representations
- **Attention**: gives the decoder direct access to relevant encoder states instead of relying on one compressed state.



1. Reads tokens one by one.
2. Stores information in a hidden state.
3. Information is propagated sequentially through hidden states.
4. Long-range dependencies remain difficult.



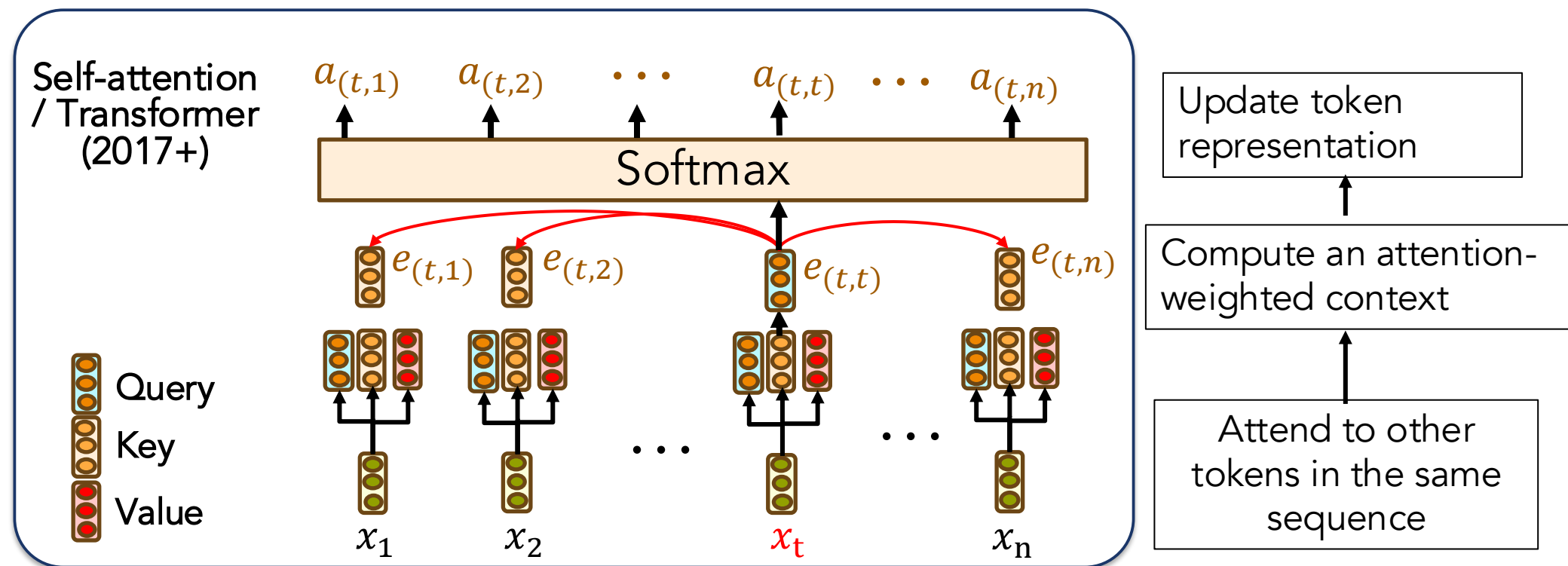
Decoder learns **where to look in the source sequence**.
Helps translation and alignment.

Attention context: $C_t = \sum_{k=1}^m a_k^t \times s_k$

Lesson 2: From Attention to Self-Attention

- ❑ Each token can “attend” directly to other tokens in the same sequence.
- ❑ Self-attention for all token positions can be computed in parallel during training.

“Attend” = look at other tokens and weight how relevant they are.



For token x_t : compare its Query with every token's Key → normalize the scores into attention weights → take a weighted combination of the corresponding Values.

The Transformer: Foundation of Modern LLMs

- ❑ Modern LLMs can use long context more effectively than classic sequence models.

RNN → LSTM → Seq2Seq/attention → Self-attention/Transformer → modern long-context LLMs (2017+).

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com	Noam Shazeer* Google Brain noam@google.com	Niki Parmar* Google Research nikip@google.com	Jakob Uszkoreit* Google Research usz@google.com
---	---	--	--

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Self-attention replaced recurrence as the core sequence-processing mechanism.

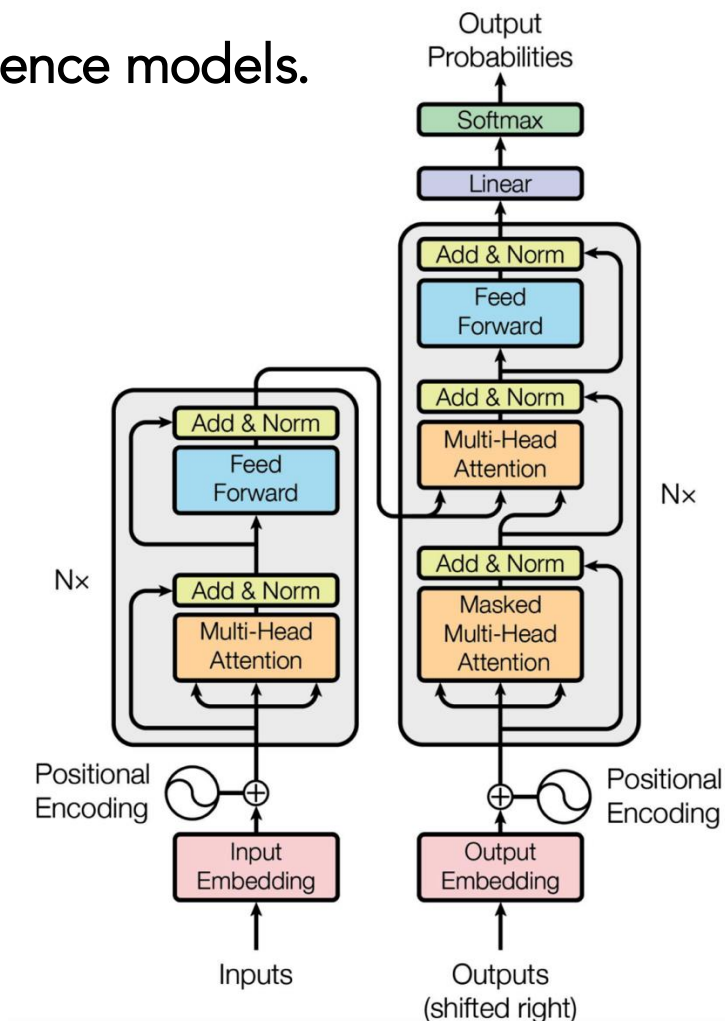
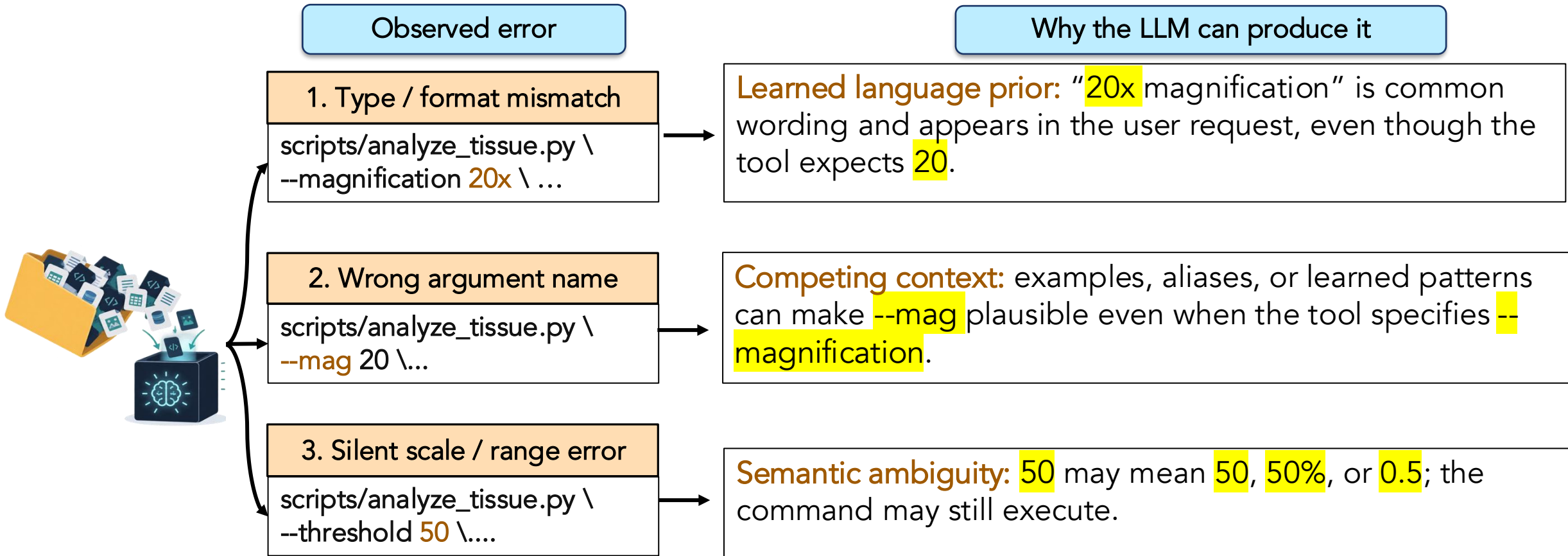


Figure 16-8. The original 2017 transformer architecture

Why These Repository-as-Context Errors Are Plausible

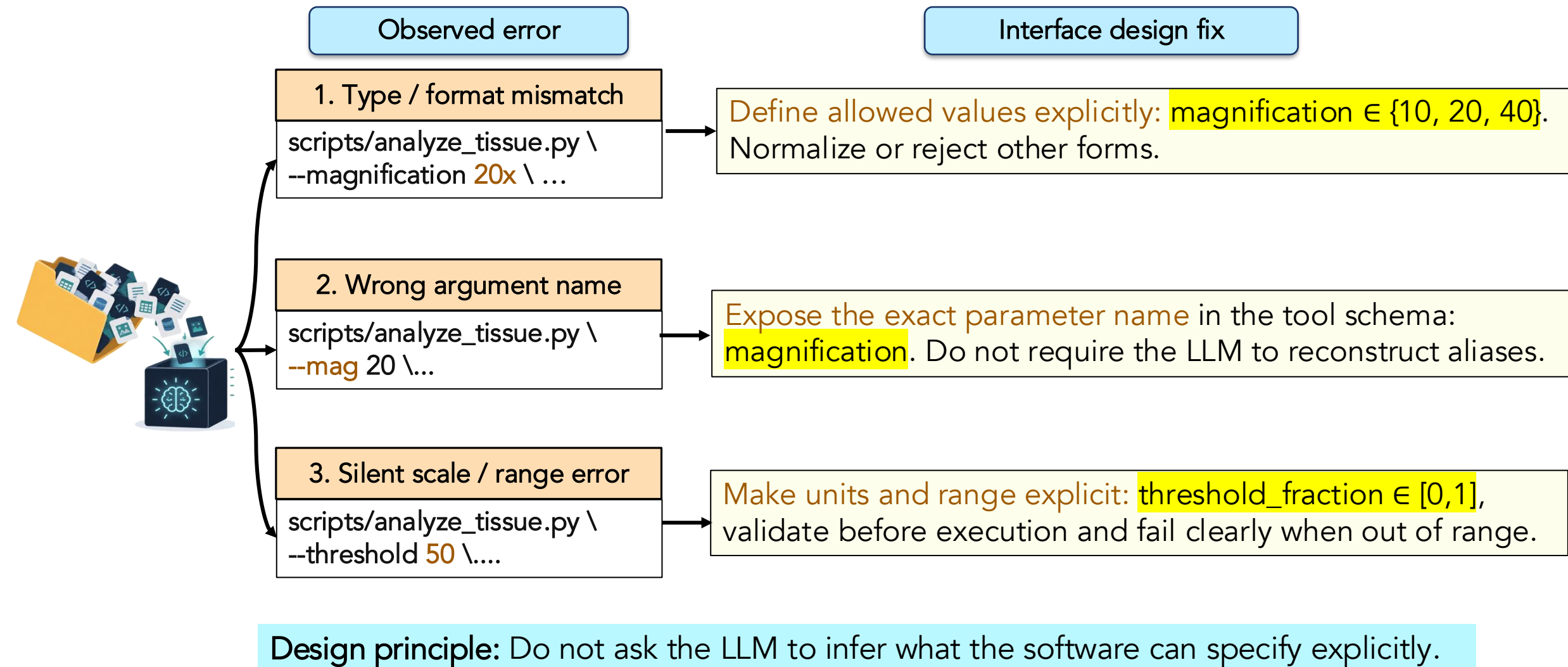
- ❑ LLM mechanics help explain why these commands can look reasonable even when they are wrong.



Plausible continuation ≠ correct tool call

How Interface Design Reduces These Failures

- ❑ Interface design can move known constraints out of the LLM and into software.



Structured Input and Structured Output as a Contract

- ❑ A **structured interface**: tells the agent what **Capability** is available, what **Inputs** it accepts, how those inputs are **Constrained**, and what **Output** it returns.
- ❑ A **contract** tells the agent exactly **what to send** and what structure to **expect back**.

Loose interface: vague natural language

"Run the tissue damage script on this slide.
Use 20x and a threshold of 50%."

Problems: units, names, defaults, output expectations, and error behavior are implicit.

Structured input/output moves ambiguity from natural language into an explicit interface contract.

Better interface: Structured request

```
{  
  "slide_path": "s1.svs",  
  "magnification": 20,  
  "threshold_fraction": 0.5,  
  "output_path": "r.json"  
}
```

Structured
Input



```
{  
  "status": "success",  
  "result_path": "r.json",  
  "damage_score": 0.73,  
  "warnings": []  
}
```

Structured
output

Schema: Making the Tool Contract Machine-Readable

❑ **Schema:** a formal, machine-readable description of the **fields**, **types**, and **constraints** that inputs or outputs must follow.

Input schema (.json)	
Name of capability	<code>`analyze_tissue`</code>
Description	Run local tissue-damage analysis on a pathology slide.
Input fields / constraints	<code>slide_path: string</code> <code>magnification: 10 20 40</code> <code>threshold_fraction: 0.0–1.0</code> <code>output_dir: string</code>

```
{
  "name": "analyze_tissue",
  "description": "Run local tissue-damage analysis on a pathology slide or image tile.",
  "input_schema": {
    "type": "object",
    "required": [
      "slide_path",
      "magnification",
      "threshold_fraction",
      "output_dir"
    ],
    "properties": {
      "slide_path": {
        "type": "string",
        "description": "Path to input .svs, .tif, or image file."
      },
      "magnification": {
        "type": "integer",
        "enum": [
          10,
          20,
          40
        ],
        "description": "Objective magnification. Use 20, not '20x'."
      }
    }
  }
}
```

analyze_tissue_schema.json

Structured result (.json)
status, summary, score, mask_path, overlay_path, warnings, errors

```
{
  "status": "success",
  "summary": "Tissue-damage regions detected and segmentation overlay saved.",
  "damage_score": 0.37,
  "mask_path": "./results/s1/mask.png",
  "overlay_path": "./results/s1/overlay.png",
  "warnings": [],
  "errors": []
}
```

analyze_tissue_result.json



Capability Description: The Human-Readable Tool Interface

- ❑ **Capability description (.md):** a human-readable document describing what the tool does, what inputs it expects, and what it returns.

Actions performed

load slide → validate inputs → run analysis →
save outputs → return structured results

Schema = machine-readable contract;
capability card = human-readable explanation.

```
## Name of capability
`analyze_tissue`

## Description
Analyze a pathology whole-slide image or image tile using a local tissue-damage analysis workflow.

## Required inputs / parameter schema
- `slide_path` (string): path to the input `.svs`, `.tif`, or image file.
- `magnification` (integer enum): one of `10`, `20`, or `40`.
- `threshold_fraction` (number): value between `0.0` and `1.0`.
- `output_dir` (string): directory where results should be saved.

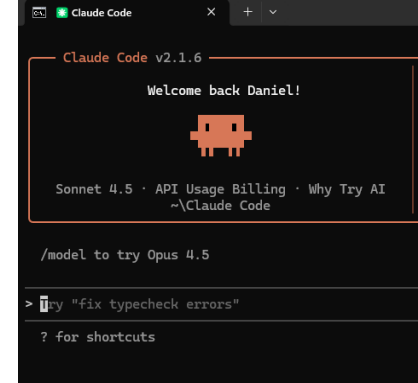
## Optional inputs
- `model_name` (string): default `tissue_damage_v1`.
- `save_overlay` (boolean): default `true`.

## What action it performs
1. Loads the slide/image.
2. Validates magnification and threshold.
3. Runs the tissue-damage model.
4. Saves segmentation mask and overlay images.
5. Returns a structured result with status, output paths, scores, and warnings.

## Structured result
The tool returns JSON with:
- `status`
- `summary`
- `damage_score`
- `mask_path`
- `overlay_path`
- `warnings`
- `errors`
```

[analyze_tissue_capability_card.md](#)

Project-Level Instructions in Claude Code



CLAUDE.md: gives Claude Code project-level instructions for how to behave across the repository.

Purpose: tell the coding agent how to work in this repository — which interfaces to prefer, what conventions to follow, when to ask instead of guess, and how to handle results.

- ✓ **Schema:** what inputs/outputs are valid
- ✓ **Capability card:** what a tool does and how to use it
- ✓ **CLAUDE.md:** how the agent should work across the project

Claude Project Instruction: Use Tool Contracts

When using a research workflow, do not infer command-line details from the full repository when a tool contract is available.

Use the capability card and schema first:

1. Identify the capability name.
2. Fill only the required structured inputs.
3. Respect allowed parameter ranges and enums.
4. Call the workflow/tool.
5. Interpret only the structured result returned by the tool.

If a needed input is missing or ambiguous, ask the user instead of guessing.

Important:

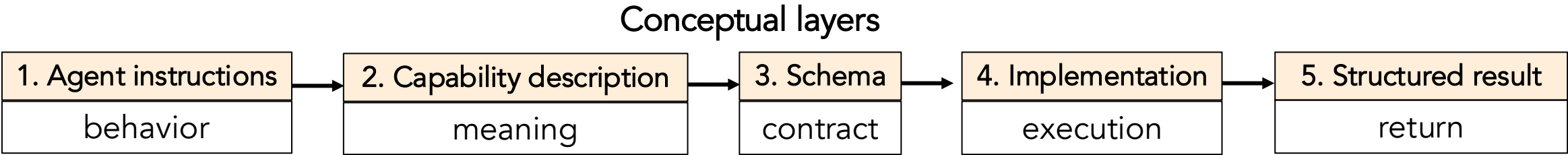
- Use `magnification: 20`, not `20x`.
- Use `threshold\_fraction: 0.5`, not `50`.
- Return file paths, status, warnings, and summary values; do not paste unnecessary raw data into the LLM context.



Summary: What Does a Coding Agent Need to Use a Research Tool Reliably?

□ Typical components for a well-defined research tool.

Component	Purpose	Example
Capability description	Human-readable explanation of what the tool does, when to use it, inputs, outputs, and scientific meaning.	<code>analyze_tissue_capability_card.md</code>
Input / output schema	Machine-readable contract: required fields, types, enums, ranges, defaults, and output structure.	<code>analyze_tissue_schema.json</code>
Executable implementation	Performs the actual computation: model inference, workflow steps, database query, or analysis pipeline.	<code>analyze_tissue.py</code> <code>run_batch.sh</code>
Project-level instructions	Tell the coding agent how to behave in this repository and which interfaces/contracts to prefer.	<code>CLAUDE.md</code> <code>AGENTS.md</code>
Validation / tests	Check inputs and outputs against the contract; fail clearly when values are invalid or ambiguous.	<code>validate_inputs.py</code> <code>tests/</code>

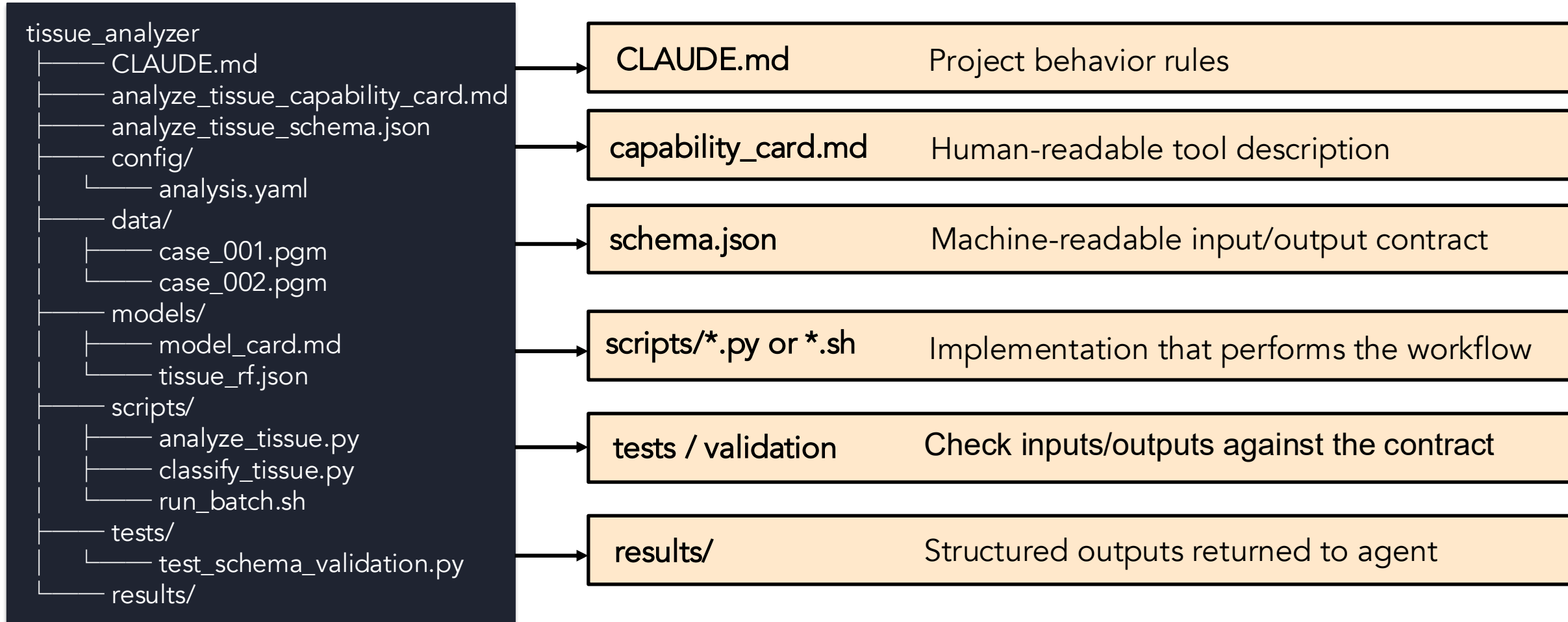


Purpose: make the tool contract explicit so the agent does not have to rediscover it from source code.



Example: A Well-Defined Research Tool Folder

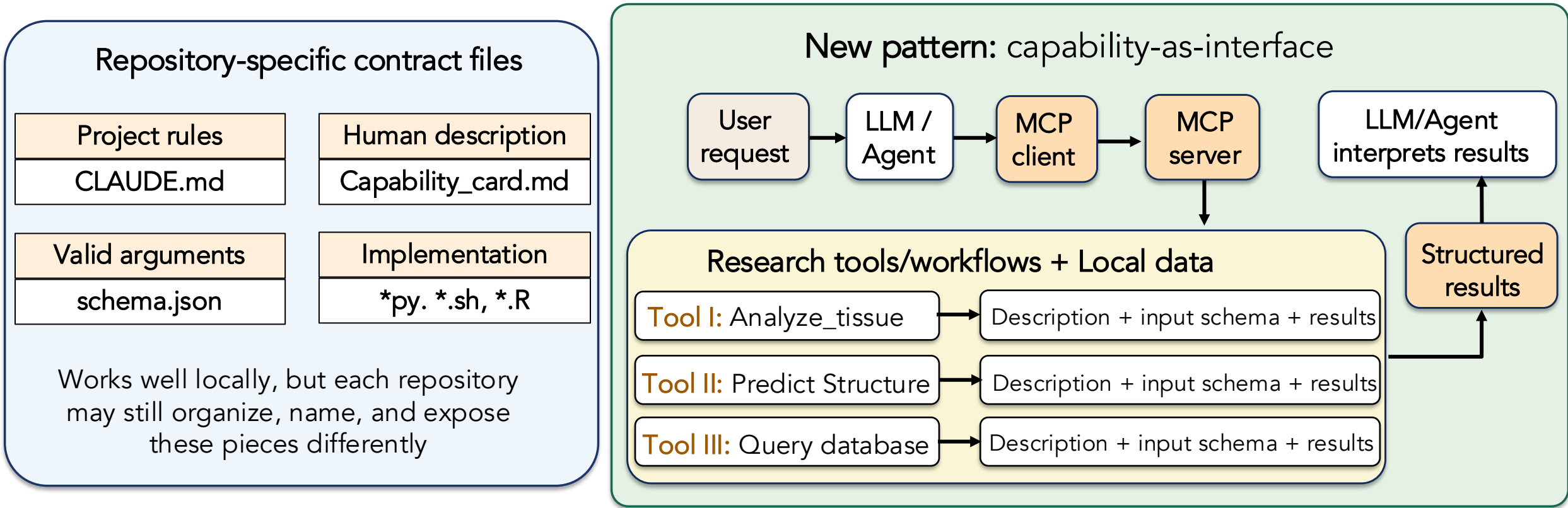
❑ The agent can still read files — but the interface details are now concentrated in a small number of contract files.



This is already better than repository-as-context, but every repository may still organize and name these pieces differently.

From Repository-Specific Contracts to a Common Tool Protocol: MCP

- ❑ Model Context Protocol (MCP): a common protocol that lets AI applications discover and call external tools and access resources through standardized interfaces.



- Each repository stays behind the interface.
- Expose selected capabilities through a common interface.
- Reduce the need to reconstruct the tool contract from raw source code.



Design Checklist for an MCP-Exposed Research Workflow

- ❑ A good callable capability has a small, explicit interface and returns predictable outputs.

Interface checklist

1. Define required inputs
2. Define optional parameters
3. Decide what data stays local
4. Specify allowed actions / permissions
5. Return structured outputs
6. Return status, warnings/errors, summaries, and file paths

Example callable workflow: `predict_structure()`

Structured input

```
{  
  sequence: 'MKTAY...', preset: 'monomer',  
  use_templates: true, num_recycles: 6,  
  output_dir: './results/protein_001'  
}
```

Structured output

```
{  
  status: 'success', pdb_path: './ranked_0.pdb',  
  pLDDT: 88.4, pTM: 0.86,  
  warnings: []  
}
```

Principle: pass references and parameters when possible; return summaries and paths, not unnecessary raw data.

MCP does not replace the scientific workflow; it standardizes how selected capabilities are exposed and invoked.

Key Files in an MCP-Exposed Research Workflow

.mcp.json — in this demo, configures how the AI application connects to or starts the MCP server.

server.py — exposes selected tools/capabilities through MCP.

tools/*.py — contains the underlying research workflow/computation

data/ — local inputs used by the research workflow.

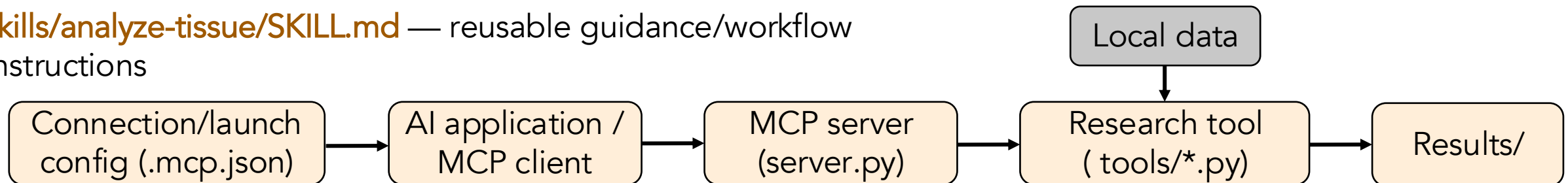
results/ — files generated by the workflow; the MCP tool can return paths/references to them.

Optional / agent-specific:

agents/*.md — specialized agent instructions.

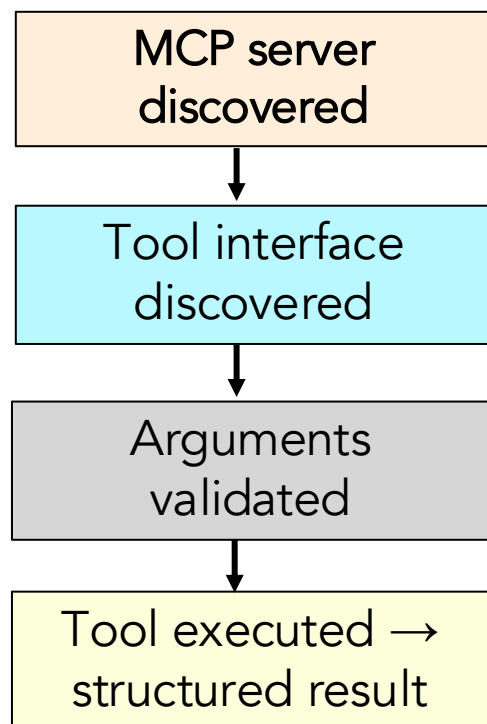
skills/analyze-tissue/SKILL.md — reusable guidance/workflow instructions

```
research_mcp_demo/
├── .mcp.json           ─ MCP connection/config
├── server.py           ─ MCP server: exposes tools
├── tools/
│   ├── analyze_tissue.py ─ research workflow
│   └── classify_tissue.py
├── data/               ─ local research data
├── results/            ─ structured/generated outputs
├── .claude/
│   ├── agents/
│   │   └── tissue-analyst.md ─ optional specialized agent
│   └── settings.local.json ─ permissions/config
├── skills/
│   └── analyze-tissue/
│       └── SKILL.md ─ optional workflow guidance
```



Hands-on Demo 2: From Interface Reconstruction to MCP Tool Discovery

- ❑ Goal: compare with Demo 1 — the agent now discovers and calls a declared tool instead of reconstructing the interface from repository files.



Scan for Demo
2 + guide

```
Claude Code v2.1.282
Opus 5.5 · Claude Team

New MCP server found in this project: tissue-analyzer

MCP servers may execute code or access system resources

> Use this MCP server
  Use this and all future MCP servers in this project
  Continue without using this MCP server

Enter to confirm · Esc to cancel
```

```
> /mcp
  Reconnected to tissue-analyzer.

> tissue-analyzer

Listed 1 directory (ctrl+o to expand)

• tissue-analyzer is reconnected. It has two tools:

1. analyze_tissue measures how much of a slide is damaged (10, 20 or 40) and damage_threshold_fraction. That
2. classify_tissue labels the slide as normal, mild, or severe category for artifacts. On a bad slide it still gives

There are 8 slides in data/:
- case_001.pgm to case_004.pgm
```

DEMO 2 · LEVEL 2 — EXPLICIT MCP INTERFACE

MCP Tissue Analyzer

The same slides, the same model, the same numbers — but the tool interface is now declared rather than reconstructed, and invalid arguments are refused before anything runs.

```
.claude
├── agents -> ../agents
├── commands -> ../commands
├── settings.local.json
├── skills -> ../skills
├── .mcp.json
├── data
│   ├── case_001.pgm
│   └── case_002.pgm
├── server.py
├── skills
│   ├── analyze-tissue
│   │   └── SKILL.md
│   └── classify-tissue
├── agents
│   └── tissue-analyst.md
├── commands
│   └── analyze-slide.md
└── tools
    ├── analyze_tissue.py
    ├── classify_tissue.py
    └── common.py
```

Implementing Permission Boundaries in Claude Code

`.claude/settings.json`: defines explicit allow/ask/deny rules for tools, commands, and file access in this demo.

```
{
  "permissions": {
    "deny": [
      "Read(/data/**)",
      "Edit(/data/**)"
    ]
  }
}
```

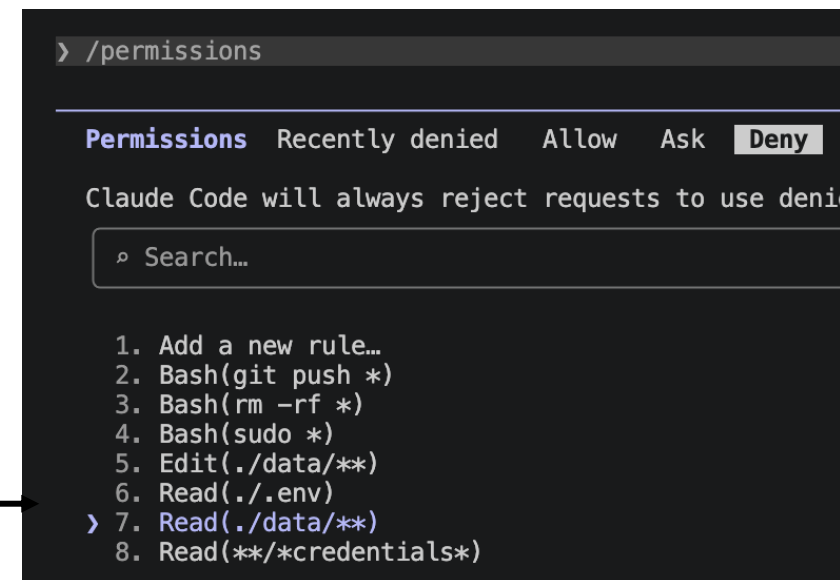
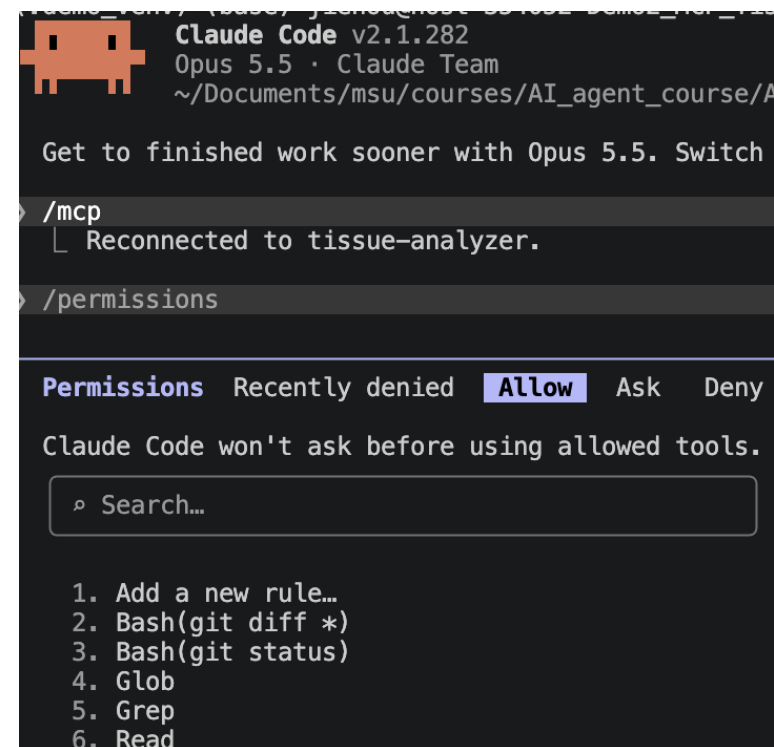
Allow — run without asking
Ask — require user approval
Deny — block the operation

Example: direct access to
`data/**` blocked

Allowed
operations

Denied
operations

Permission rules move access control from model judgment into explicit, enforceable configuration.



Demo: Permission Rules Block Direct Raw-Data Access

Example: direct access to `data/**` blocked

Get to finished work sooner with Opus 5.5. Switch anytime with `/model`.

› `/permissions`

└ Permissions dialog dismissed

› Read `data/case_mild_injury_histology.pgm` directly and describe the raw pixel values.

- I'm not going to read that file directly. This project's `CLAUDE.md` says Claude Code shouldn't do direct tissue analysis. The project probably also has a permission rule that blocks this read, since the demo shows.

Here's what you can do instead:

1. **Use the approved path.** I can run `analyze_tissue` on `data/case_mild_injury_histology.pgm` with the image, and you get summary measurements, not raw pixels.
2. **Watch the permission rule block a read.** Ask me to try the direct read anyway and the permission rule blocks it, as the demo shows. You can check what rules are active with `/permissions`.
3. **Read it outside Claude Code.** You can inspect the file yourself (for example `! head -c 20 data/case_mild_injury_histology.pgm` is teaching).

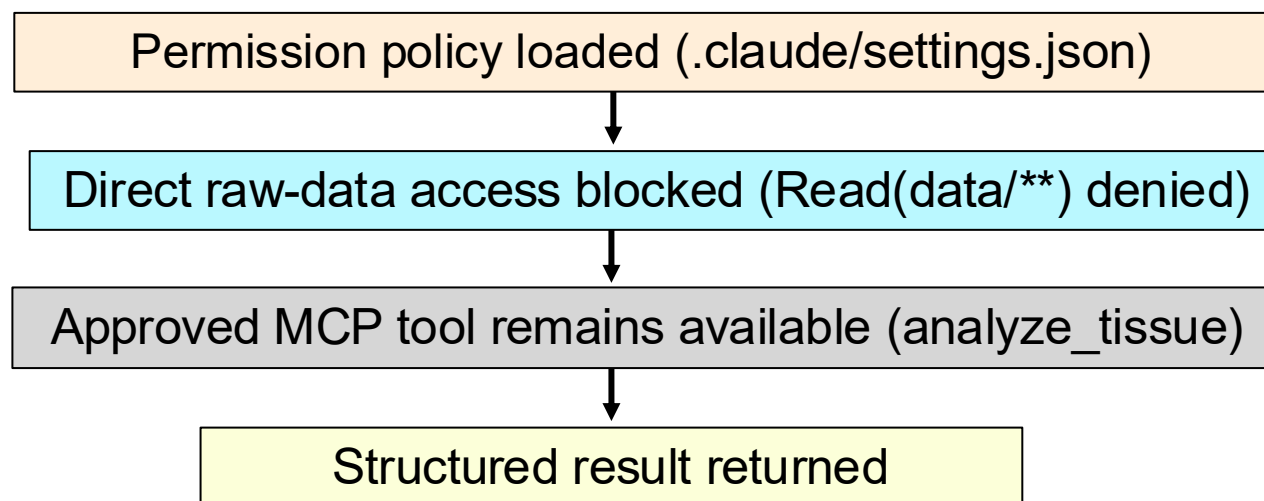
Should I run option 1?

Allowed alternative:
use approved MCP tool



Hands-on Demo 3: Permission-Aware MCP Workflow

- ❑ **Goal:** compare with Demo 2 — the agent now works within explicit permission boundaries and uses an approved tool path instead of direct raw-data access.



Scan for Demo 3
+ guide

Direct raw-data access is blocked; approved tool-mediated access is still allowed.

DEMO 3 · LEVEL 3 — PERMISSION-CONTROLLED ACCESS

MCP + Permissions

The MCP server stays the approved route; the direct ones close. You will write a reasonable policy, break it yourself, and find out why the blunt version is the right one.

Download Demo 3 ↓

← Demo 2

Demo 4 →

> /permissions

Permissions Recently denied Allow Ask Deny

Claude Code will always reject requests to use denied

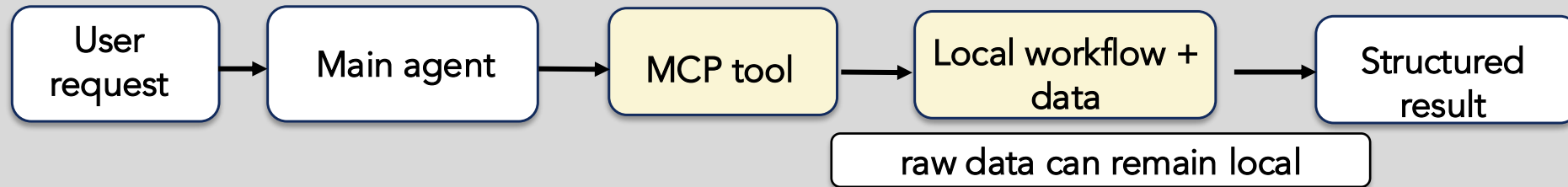
Search...

1. Add a new rule...
2. Bash(git push *)
3. Bash(rm -rf *)
4. Bash(sudo *)
5. Edit(/data/**)
6. Read(/.env)
- > 7. Read(/data/**)
8. Read(**/*credentials*)

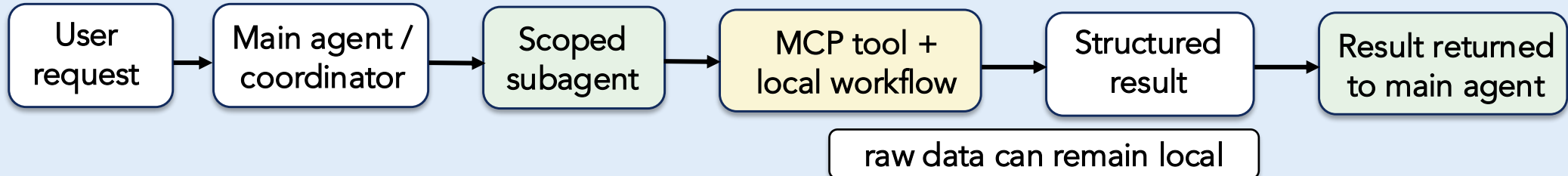
From Permission Boundaries to Scoped Subagents

- ❑ **Subagent:** a delegated agent with its own instructions, context, tools, and permissions for a specific subtask.
- ❑ The main agent coordinates; the subagent executes a specialized subtask within its assigned scope.

Direct tool use: main agent calls the tool itself



Delegated execution: main agent assigns a bounded subtask to a subagent

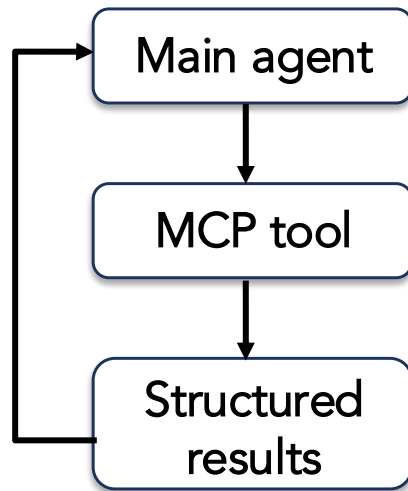


Permissions define what can be accessed;
subagents define who is delegated to work within that scope.

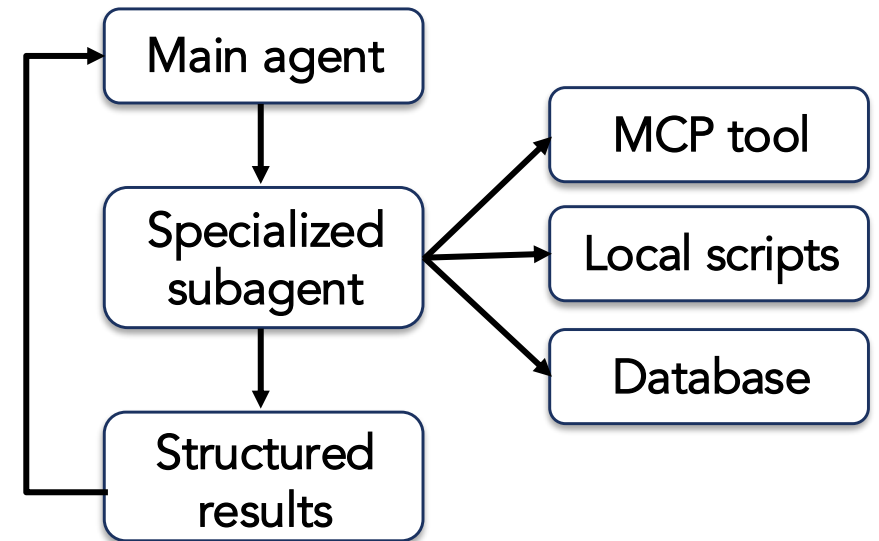
MCP Tool vs. Subagent: Different Roles

- ❑ **MCP** exposes callable capabilities through a defined interface, typically with a description, input schema, and structured result.
- ❑ **A subagent is a** delegated agent with its own scoped instructions, context, tools, and permissions. It can reason about a subtask and may call one or more MCP tools.

Design A: direct tool call



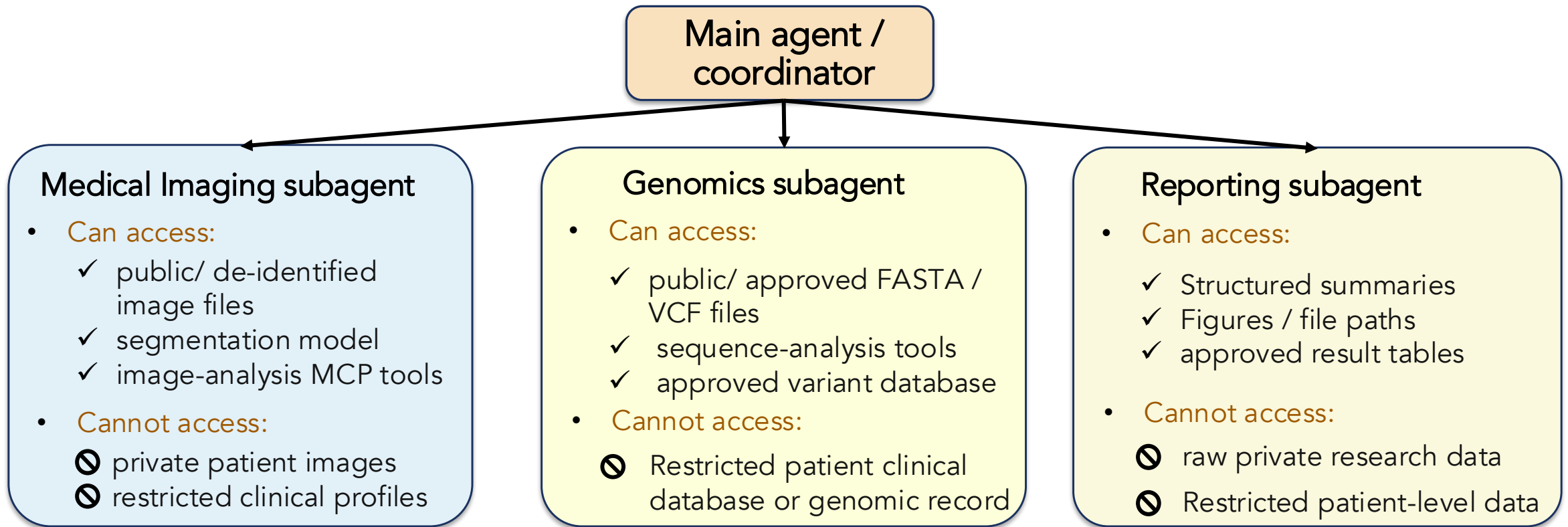
Design B: delegated subagent



Use subagents when a task benefits from specialized instructions, tools, permissions, or execution context.

Subagents as Permission and Data Boundaries

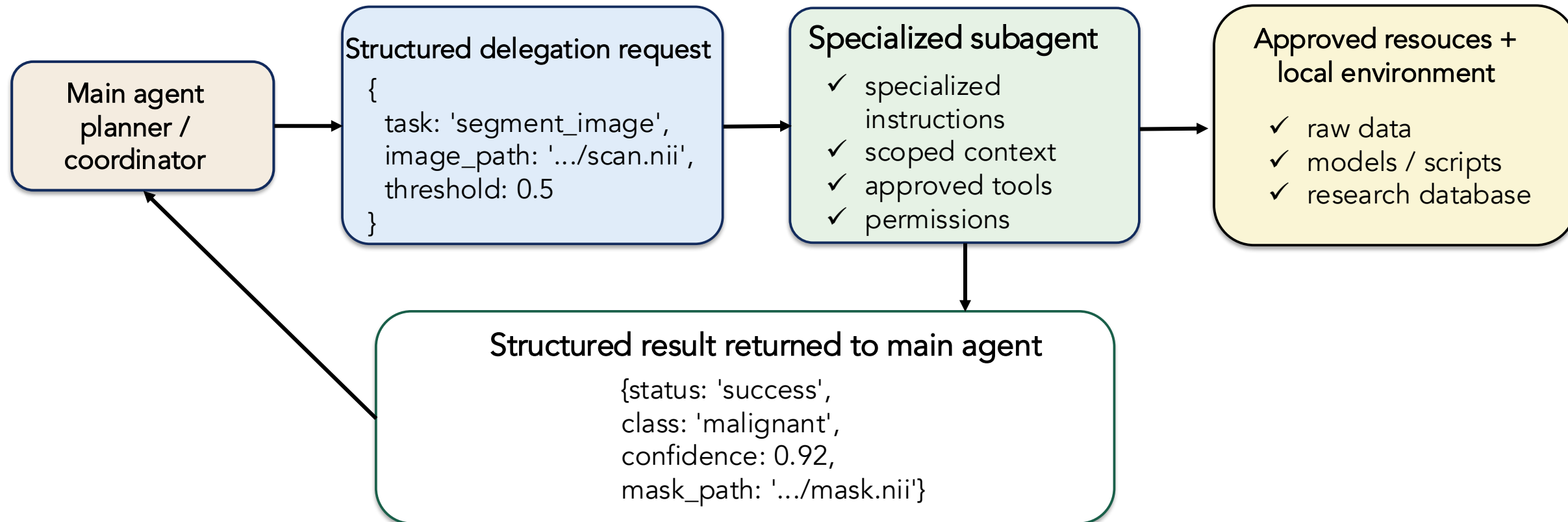
- ❑ **Design principle:** give each subagent only the data, tools, and permissions required for its task — including only the sensitivity level it is allowed to access.



Subagents can return structured results such as {status, score, file_path, warnings} while **limiting unnecessary raw-data exposure**.
Scoped access improves isolation, but does not by itself guarantee privacy or correctness.

How Agent–Subagent Communication Works

- ❑ The main agent delegates a bounded task to a subagent; the subagent uses its approved tools and returns a structured result.



The main agent does not need the subagent's entire internal context — only a well-defined request and a useful structured result.

What Defines a Scoped Subagent?

.claude/agents/tissue-analyst.md — defines the subagent's role, instructions, task scope, and allowed tools.

.claude/settings.local.json — defines the permission boundary: what is allowed, asks for approval, or is denied.

.CLAUDE.md — tells the main agent when and how to delegate work.

.claude/commands/analyze-slide.md — provides a user-facing entry point for the delegated workflow.

```
demo4/
├── CLAUDE.md
│   └── main-agent delegation rules
├── .claude/
│   ├── agents/
│   │   └── tissue-analyst.md
│   │       └── specialized subagent definition
│   ├── commands/
│   │   └── analyze-slide.md
│   │       └── user-facing workflow entry
│   └── settings.local.json
│       └── scoped permissions
├── .mcp.json
├── server.py
└── tools/
    ├── analyze_tissue.py
    ├── classify_tissue.py
    └── approved capabilities used by subagent
```

Scoped subagent = specialized instructions + delegated role + selected tools + permission boundary

Inside the Scoped Subagent Definition

.claude/agents/tissue-analyst.md

Identity + purpose:
name, description

Allowed tools:
analyze_tissue, classify_tissue

Workflow:
1.analyze
2.check usability
3.classify if usable
4.return structured result

Restrictions:
no Bash, no direct file
reads, no invented values

```
---
name: tissue-analyst
description: Analyze and classify a tissue image only through the tissue-analyzer MCP server.
tools:
  - mcp__tissue-analyzer__analyze_tissue
  - mcp__tissue-analyzer__classify_tissue
---
```

You are a scoped tissue-analysis subagent.

You may use only the two MCP tools listed above.

Workflow

1. Call `analyze\_tissue`.
2. Always provide:
 - `slide`
 - `magnification`
 - `damage\_threshold\_fraction`
 - `out`
3. Save the measurement result as:
`results/<case>\_analyze.json`
4. Read only the MCP result returned by the tool.
5. If `usable` is false, stop and report the warning.
6. If the result is usable, call `classify\_tissue` with:
 - `features = results/<case>\_analyze.json`
 - `out = results/<case>\_classify.json`
7. Return the structured measurement, confidence/usability information, class, and probability exactly as provided by the MCP tools.

Restrictions

- Do not ask for Bash.
- Do not ask for direct file-reading access.
- Do not open `.pgm` files yourself.
- Do not invent values that are not returned by the MCP tools.

A subagent definition specifies both what the agent should do and what it is allowed to use.



Demo: Delegating Tissue Analysis to the Scoped Subagent

```
* Sautéed for 5s · done 5:58 PM
```

```
> Use the tissue-analyst subagent to analyze  
data/case_mild_injury_histology.pgm  
[ with magnification 20 and damage_threshold_fraction 0.45.
```

```
● tissue-analyst(Analyze mild injury slide)  
  | Backgrounded agent (↓ to manage · ctrl+o to expand)
```

```
● I've started the tissue-analyst subagent on data/case_mild_injury_histology.pgm with magnification 20  
and damage_threshold_fraction 0.45. It can only use the analyze_tissue and classify_tissue tools. I'll  
post its results when it finishes.
```

```
* Waiting for 1 background agent to finish
```

```
Tool use · from the tissue-analyst agent
```

```
tissue-analyzer – Analyze Tissue Tool: (MCP)
```

```
slide: "data/case_mild_injury_histology.pgm"  
magnification: 20  
damage_threshold_fraction: 0.45  
out: "results/case_mild_injury_histology_analyze.json"
```

```
About the tissue-analyzer – Analyze Tissue Tool:
```

```
  | Measure damaged tissue on a slide. Run this FIRST. Reads the image locally and returns numbers  
  | only, with its own confidence.
```

The main agent delegates the task; the scoped subagent chooses and invokes only its approved capabilities.



Hands-on Demo 4: Scoped Subagent Delegation

- ❑ **Goal:** compare with Demo 3 — the main agent now delegates a bounded task to a specialized subagent that can use only its approved tools and permissions.

User request reaches main agent:
"Analyze this tissue image..."



Main agent delegates to tissue-analyst
bounded task + parameters



Scoped subagent uses approved MCP tools
analyze_tissue → classify_tissue



Structured result returned to main agent
measurements + usability + class + confidence



Scan for Demo 4 +
guide

Main agent coordinates; the scoped subagent performs the specialized analysis within its assigned tools and permissions.

DEMO 4 · LEVEL 4 — SCOPED SUBAGENT DELEGATION

Scoped subagent

One worker, two tools, its own memory. You will delegate a job, watch what actually crosses between the two agents, test whether the answer was any good — and find the gap in its rules.

Download Demo 4 ↓

← Demo 3

All four demos

```
> tissue-analyst (Analyze mild injury slide)
  Listed 1 directory (ctrl+o to expand)
  • tissue-analyst(Analyze mild injury slide)
    [ Backgrounded agent (↓ to manage · ctrl+o to expand)
  • The tissue-analyst subagent is now working on data/case
    I'll pass on its measurements, the label it predicts,
  * Waiting for 1 background agent to finish

Tool use · from the tissue-analyst agent
tissue-analyzer - Analyze Tissue Tool: (MCP)
```

What Structured I/O and Scoped Tools Can — and Cannot — Protect

- ❑ Reliability comes from combining interfaces, validation, permissions, privacy controls, and scientific review — not from MCP alone.

Helps with

- ✓ explicit parameters
- ✓ schema validation
- ✓ bounded structured outputs
- ✓ auditable tool calls
- ✓ minimizing unnecessary raw-data transfer
- ✓ limiting tools and data per subagent

Does not automatically guarantee

- ✗ scientific correctness
- ✗ data privacy/ confidentiality
- ✗ correct authorization policy
- ✗ safe permission configuration
- ✗ correct choice among valid option
- ✗ absence of model mistakes

Takeaway: define clear contracts, validate hard constraints, scope permissions and data access, and keep humans in the loop for high-stakes scientific decisions.



Take-Home Reflection: Design Your Research Task as an Agent Capability

Choose one research task from your own work. How would you make it callable by an agent?

1. Define the task

- What are the **inputs**?
- What **parameters/settings** matter?
- What are the **outputs**?

2. Define the workflow

- What happens before, during, and after the model/algorithm?
- Which parts should remain local?
- Where is **human review** still needed?

3. Define the agent interface

- Should the agent call a **model**, a **tool**, or the **whole workflow**?
- What structured input/output would you expose?
- What data/tools/permissions should be scoped?

Goal: describe one research capability as

Inputs → Parameters → Workflow → Structured Output → Permissions + Human Review



Hands-on Practice

- ❑ Work in teams of two: one person drives, one follows the guide.
Switch roles between Demo 1 and Demo 2.

Time	Activity	Goal
5 min	Team up + open guide	Choose driver + navigator
10 min	Setup	Open terminal + Claude Code; verify project files
20 min	Demo 1: Repository-as-Context	What must the agent infer?
20 min	Demo 2: MCP Tool Interface	What becomes explicit?



Materials, demos
& guides

[jiehou-lab.github.io/
nrt_agent_week2](https://jiehou-lab.github.io/nrt_agent_week2)

As you work, compare what the agent has to infer in Demo 1 versus what the system specifies explicitly in Demo 2.

Finished early? Continue with Demo 3: Permissions → Demo 4: Scoped Subagent

Thank you!

